



# Git & GitHub: Do Zero ao Avançado

Curso completo com 9 módulos

Configuração • Branches • Colaboração • Rebase • Actions • Segurança

CloudTrilhas — [cloudtrilhas.com.br](https://cloudtrilhas.com.br)

Por Ricardo Simines Scopim

 **Git & GitHub — Módulo 01**

## Fundamentos

git init, add, commit, push, pull, clone, status, log, .gitignore. O fluxo básico do Git: inicializar, rastrear mudanças, criar snapshots e sincronizar com o GitHub.

## 1. Criando e Clonando Repositórios

### git init CRIAR

Inicializa um repositório Git local.

```
mkdir meu-projeto
cd meu-projeto
git init
# Initialized empty Git repository
```

### git clone CRIAR

Clona um repositório remoto para sua máquina.

```
# Via SSH (recomendado)
git clone git@github.com:user/repo.git

# Via HTTPS
git clone https://github.com/user/repo.git

# Clone com nome customizado
git clone git@github.com:user/repo.git meu-dir
```

## 2. Fluxo Básico: add → commit → push

Os 3 estados do Git:

```
Working Directory → Staging Area → Repository
(modificado)      (preparado)      (commitado)
git add →         git commit →     git push → Remote
```

## git status **ESSENCIAL**

Mostra o estado atual dos arquivos.

```
git status
# On branch main
# Changes not staged for commit:
#   modified: index.html
# Untracked files:
#   style.css

git status -s # formato curto
# M  index.html
# ?? style.css
```

## git add **STAGING**

Adiciona arquivos à staging area.

```
git add arquivo.txt      # arquivo específico
git add .                 # tudo no diretório
git add *.js              # por extensão
git add -p                # interativo (por hunks)

# Remover do staging (sem perder mudanças)
git restore --staged arquivo.txt
```

**git commit** **ESSENCIAL**

Cria um snapshot das mudanças no staging.

```
git commit -m "feat: adiciona navbar"
git commit -am "fix: corrige typo" # add + commit
git commit --amend # edita último commit

# Boas práticas de mensagem:
# tipo: descrição curta (max 72 chars)
# feat: | fix: | docs: | refactor: | test:
```

**git log** **HISTÓRICO**

Visualiza o histórico de commits.

```
git log # completo
git log --oneline # resumido
git log --oneline --graph # com branches
git log -5 # últimos 5
git log --author="João" # por autor
git log --since="2024-01-01"
```

## 3. Sincronização com Remoto

---

**git remote** **REMOTO**

Gerencia conexões com repositórios remotos.

```
git remote add origin git@github.com:user/repo.git
git remote -v # lista remotos
git remote rename origin upstream
git remote remove upstream
```

**git push** ENVIAR

Envia commits locais para o remoto.

```
git push origin main
git push -u origin main    # define upstream
git push                   # após -u, basta isso
git push --force-with-lease # force seguro
```

**git pull** RECEBER

Baixa e integra mudanças do remoto.

```
git pull                # fetch + merge
git pull --rebase       # fetch + rebase
git pull origin main    # branch específica
```

**git fetch** RECEBER

Baixa mudanças sem integrar (mais seguro).

```
git fetch origin
git fetch --all        # todos os remotos
git log origin/main..main # ver diferenças
# Depois: git merge origin/main
```

## 4. Arquivos Ignorados

---

## .gitignore CONFIGURAÇÃO

Define quais arquivos o Git deve ignorar.

```
# .gitignore - exemplo para Node.js + Terraform
node_modules/
dist/
.env
.env.local
*.log

# Terraform
.terraform/
*.tfstate
*.tfstate.backup
*.tfvars

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db
```

 **Git & GitHub — Módulo 02**

## Branches: Ramificação e Merge

Criar, mergear, stash, convenções de nomes, estratégias de merge. Branches permitem trabalhar em funcionalidades isoladas sem afetar o código principal.

### 1. Criando e Navegando entre Branches

**git branch / checkout / switch** ESSENCIAL

Gerencia branches locais e remotas.

```
# Listar branches
git branch          # locais
git branch -a       # locais + remotas
git branch -v        # com último commit

# Criar branch
git branch feature/login
git checkout -b feature/login # cria e muda
git switch -c feature/login    # forma moderna

# Mudar de branch
git checkout main
git switch main

# Deletar branch
git branch -d feature/login # seguro (merged)
git branch -D feature/login # forçado
git push origin --delete feature/login # remota
```

**Convenções de nomes:** feature/ nova funcionalidade, fix/ ou bugfix/ correção, hotfix/ correção urgente em produção, release/ preparação de release, chore/ tarefas de manutenção.

### 2. Merge e Estratégias

**git merge** INTEGRAÇÃO

Integra mudanças de uma branch em outra.

```
# Merge da feature na main
git checkout main
git merge feature/login

# Merge com commit explícito
git merge --no-ff feature/login

# Abortar merge com conflito
git merge --abort
```

**git stash** UTILIDADE

Salva mudanças temporariamente sem commit.

```
# Guardar mudanças
git stash
git stash push -m "WIP: navbar"

# Listar e recuperar
git stash list
git stash pop          # aplica e remove
git stash apply stash@{0} # aplica sem remover

# Limpar
git stash drop stash@{0}
git stash clear
```

## git diff **COMPARAÇÃO**

Mostra diferenças entre estados.

```
git diff           # working vs staging
git diff --staged  # staging vs commit
git diff main..feature # entre branches
git diff HEAD~3    # últimos 3 commits
git diff --stat    # resumo
```

## Estratégias de Merge **CONCEITO**

Entenda quando usar cada tipo.

```
# Fast-forward (padrão quando possível)
# Histórico linear, sem commit de merge
git merge feature # se main não avançou

# No fast-forward (preserva histórico)
git merge --no-ff feature

# Cria commit de merge sempre

# Squash (compacta commits)
git merge --squash feature
git commit -m "feat: login completo"
```

 Git & GitHub — Módulo 03

## Colaboração: Fork, Pull Request e Code Review

Fork, Pull Request, Code Review, Issues, Templates. Aprenda o fluxo profissional de contribuição em projetos.

### 1. Fork e Contribuição

---

## Fork Workflow COLABORAÇÃO

Fluxo completo para contribuir em projetos open source.

```
# 1. Fork no GitHub (botão Fork)

# 2. Clone do seu fork
git clone git@github.com:SEU-USER/projeto.git
cd projeto

# 3. Adicione o repositório original como upstream
git remote add upstream git@github.com:ORIGINAL/projeto.git

# 4. Crie branch para sua contribuição
git checkout -b fix/corrige-typo

# 5. Faça suas mudanças e commit
git add .
git commit -m "fix: corrige typo na documentação"

# 6. Push para seu fork
git push origin fix/corrige-typo

# 7. Abra Pull Request no GitHub (origin → upstream)

# 8. Manter fork atualizado
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

## 2. Pull Requests e Code Review

---

## Pull Request GITHUB

Boas práticas para criar PRs eficientes.

```
# Título claro e descritivo
feat: implementa autenticação OAuth2

# Descrição com template:
## O que foi feito
- Implementa login com Google
- Adiciona middleware de auth

## Como testar
1. Rodar `npm run dev`
2. Acessar /login
3. Clicar em "Login com Google"

## Screenshots
(se aplicável)

Closes #42
```

## Code Review

### BOAS PRÁTICAS

Checklist para revisar código de colegas.

# O que verificar no Code Review:

- ✓ Código funciona conforme descrito?
- ✓ Testes foram adicionados/atualizados?
- ✓ Sem código duplicado?
- ✓ Nomes de variáveis claros?
- ✓ Sem secrets hardcoded?
- ✓ Performance aceitável?
- ✓ Documentação atualizada?

# Tipos de comentário:

- #  Blocker – precisa corrigir
- #  Suggestion – melhoria opcional
- #  Question – dúvida/discussão

## Issues GITHUB

Organize trabalho com Issues e Labels.

```
# Criar Issue com template:
## Descrição do Bug
Ao clicar em "Salvar", nada acontece.

## Passos para Reproduzir
1. Acessar /settings
2. Alterar nome
3. Clicar "Salvar"

## Comportamento Esperado
Dados salvos com sucesso.

## Ambiente
- Browser: Chrome 120
- OS: Ubuntu 22.04

# Labels úteis:
# bug, enhancement, documentation
# good first issue, help wanted
```

**GitHub CLI** **PRODUTIVIDADE**

Gerencie PRs e Issues pelo terminal.

```
# Instalar: https://cli.github.com
gh auth login

# Pull Requests
gh pr create --title "feat: login"
gh pr list
gh pr checkout 42
gh pr merge 42 --squash

# Issues
gh issue create --title "Bug: login"
gh issue list --label "bug"
gh issue close 42

# Repos
gh repo create meu-projeto --public
gh repo clone user/repo
```

 **Git & GitHub — Módulo 04**

## Resolução de Conflitos

Identificar, resolver e prevenir conflitos de merge. Conflitos acontecem quando duas pessoas editam o mesmo trecho — saber resolvê-los é essencial.

### 1. Entendendo e Resolvendo Conflitos

#### Conflito de Merge **ESSENCIAL**

Anatomia de um conflito e como resolver.

```
# Quando o Git não consegue resolver automaticamente:
<<<<<< HEAD
const titulo = "Versão da branch atual";
=====
const titulo = "Versão da branch incoming";
>>>>>> feature/nova-funcionalidade

# Resolução:
# 1. Edite o arquivo – escolha uma versão ou combine
const titulo = "Versão final combinada";

# 2. Remova os marcadores (<<<<, ==, >>>>)

# 3. Adicione e commite
git add arquivo-resolvido.js
git commit -m "fix: resolve conflito no título"

# Ferramentas visuais:
git mergetool          # abre ferramenta configurada
# VS Code: aceitar current/incoming/both
```

## Prevenir Conflitos DICA

Boas práticas para minimizar conflitos.

```
# 1. Pull frequente
git pull --rebase origin main

# 2. Branches curtas (1-3 dias)

# 3. Comunicação com o time

# 4. Arquivos pequenos e focados

# 5. Rebase antes de abrir PR
git checkout feature
git rebase main

# Resolve conflitos uma vez só
```

## Abortar e Recomeçar SEGURANÇA

Quando a resolução fica complexa demais.

```
# Abortar merge em andamento
git merge --abort

# Abortar rebase em andamento
git rebase --abort

# Voltar ao estado anterior
git reset --hard HEAD

# Ver estado antes do conflito
git stash
git log --oneline --graph
```

 Git & GitHub — Módulo 05

# Histórico e Reescrita: Rebase, Cherry-pick e Reflog

Rebase interativo, cherry-pick, reflog, reset, revert. Ferramentas avançadas para manipular o histórico de commits.

## 1. Git Rebase

### git rebase AVANÇADO

Reaplica commits sobre outra base. Cria histórico linear e limpo.

```
# Rebase simples – atualiza feature com main
git checkout feature/login
git rebase main

# Seus commits são reaplicados sobre o topo da main

# Rebase interativo – editar/squash/reorder commits
git rebase -i HEAD~5

# pick abc1234 feat: adiciona form
# squash def5678 fix: ajusta validação
# reword ghi9012 docs: atualiza README
# drop jkl3456 WIP: teste temporário

# Continuar após resolver conflito
git rebase --continue

# Abortar rebase
git rebase --abort
```

 **Regra de Ouro:** Nunca faça rebase em branches públicas/compartilhadas. Rebase reescreve o histórico — use apenas em branches locais ou pessoais.

## 2. Cherry-pick e Reflog

## git cherry-pick AVANÇADO

Aplica um commit específico em outra branch.

```
# Aplicar commit específico
git cherry-pick abc1234

# Aplicar múltiplos commits
git cherry-pick abc1234 def5678

# Aplicar sem commitar (só staging)
git cherry-pick --no-commit abc1234

# Caso de uso: hotfix
git checkout main
git cherry-pick fix-commit-hash

# Aplica a correção sem trazer toda a branch
```

## git reflog RECUPERAÇÃO

Histórico de TODAS as ações — sua rede de segurança.

```
# Ver histórico completo de movimentos
git reflog

# abc1234 HEAD@{0}: commit: feat: login
# def5678 HEAD@{1}: checkout: main → feature
# ghi9012 HEAD@{2}: reset: moving to HEAD~1

# Recuperar commit "perdido"
git checkout HEAD@{2}
git branch recuperada HEAD@{2}

# Recuperar após reset --hard
git reflog
git reset --hard HEAD@{1}

# Reflog expira em 90 dias (padrão)
```

**git reset** REESCRITA

Move o HEAD para outro commit. **Cuidado com --hard!**

```
# Soft – mantém mudanças no staging
git reset --soft HEAD~1

# Mixed (padrão) – mantém no working dir
git reset HEAD~1

# Hard – DESCARTA tudo (perigoso!)
git reset --hard HEAD~1

# Desfazer último commit (mantendo mudanças)
git reset --soft HEAD~1
# Agora pode re-commitar diferente
```

**git revert** SEGURO

Cria novo commit que desfaz outro. Seguro para branches públicas.

```
# Reverter um commit (cria novo commit)
git revert abc1234

# Reverter sem commitar automaticamente
git revert --no-commit abc1234

# Reverter merge commit
git revert -m 1 merge-commit-hash

# Diferença de reset:
# reset: apaga histórico (perigoso)
# revert: adiciona ao histórico (seguro)
```



Git & GitHub — Módulo 06

## Fluxos Profissionais: Git Flow, Tags e Conventional Commits

Git Flow, GitHub Flow, Tags, Conventional Commits, SemVer. Estratégias de branching usadas por equipes profissionais.

### 1. Git Flow vs GitHub Flow

---

## Git Flow FLUXO

Modelo robusto para projetos com releases planejadas.

```
# Branches permanentes:
# main      → código em produção
# develop   → integração de features

# Branches temporárias:
# feature/* → novas funcionalidades
# release/* → preparação de release
# hotfix/*  → correções urgentes em produção

# Fluxo:
git checkout develop
git checkout -b feature/pagamento
# ... desenvolve ...
git checkout develop
git merge --no-ff feature/pagamento

# Release:
git checkout -b release/1.2.0 develop
# ... ajustes finais, bump version ...
git checkout main
git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m "Release 1.2.0"
git checkout develop
git merge --no-ff release/1.2.0
```

## GitHub Flow FLUXO

Modelo simplificado — ideal para deploy contínuo.

```
# Apenas 1 branch permanente: main (sempre deployável)

# Fluxo:

# 1. Crie branch a partir de main
git checkout -b feature/dark-mode

# 2. Faça commits incrementais
git commit -m "feat: adiciona toggle de tema"
git commit -m "feat: implementa CSS dark mode"

# 3. Abra Pull Request cedo (Draft PR)
git push -u origin feature/dark-mode
# → Abra PR no GitHub

# 4. Code Review + CI passa

# 5. Merge na main (squash ou merge commit)

# 6. Deploy automático após merge

# Quando usar cada um:
# Git Flow → releases planejadas, múltiplas versões
# GitHub Flow → deploy contínuo, SaaS, projetos ágeis
```

## 2. Tags e Versionamento

---

## git tag RELEASE

Marca pontos específicos no histórico (releases).

```
# Tag anotada (recomendada)
git tag -a v1.0.0 -m "Primeira release"

# Tag leve
git tag v1.0.0-beta

# Listar tags
git tag -l "v1.*"

# Push tags para remoto
git push origin v1.0.0
git push origin --tags    # todas

# Deletar tag
git tag -d v1.0.0
git push origin --delete v1.0.0
```

## Conventional Commits PADRÃO

Padrão de mensagens para changelog automático.

```
# Formato:
# tipo(escopo): descrição

feat: adiciona autenticação OAuth
feat(api): implementa endpoint /users
fix: corrige validação de email
fix(auth): token não expira corretamente
docs: atualiza README com exemplos
style: formata código com prettier
refactor: extrai lógica para service
test: adiciona testes para UserService
chore: atualiza dependências
ci: adiciona step de lint no pipeline

# Breaking change:
feat!: remove suporte a Node 14

# ou

feat: migra para ESM
BREAKING CHANGE: require() não funciona mais
```

## 3. Semantic Versioning (SemVer)

### MAJOR.MINOR.PATCH — Exemplo: v2.1.3

```
MAJOR (2) → Breaking changes (incompatível com anterior)
MINOR (1) → Nova funcionalidade (compatível)
PATCH (3) → Correção de bug (compatível)

Exemplos:
v1.0.0 → v1.0.1 (fix: corrige bug)
v1.0.1 → v1.1.0 (feat: nova feature)
v1.1.0 → v2.0.0 (feat!: breaking change)
```



Git & GitHub — Módulo 07

## Automação: GitHub Actions, CI/CD e Hooks

GitHub Actions, Hooks, Husky, lint-staged, pipelines. Automatize testes, builds e deploys diretamente no GitHub.

### 1. GitHub Actions — Fundamentos

---

**Workflow YAML** CI/CD

Arquivo de workflow em `.github/workflows/`.

```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build
```

## Deploy Workflow CD

Exemplo de deploy automático para AWS S3 + CloudFront.

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS Credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/github-actions
          aws-region: us-east-1

      - name: Sync to S3
        run: aws s3 sync ./dist s3://meu-bucket --delete

      - name: Invalidate CloudFront
        run: |
          aws cloudfront create-invalidation \
            --distribution-id ${ secrets.CF_DISTRIBUTION_ID } \
            --paths "/*"
```

## 2. Terraform CI com GitHub Actions

---

## Terraform Pipeline IAc

```
# .github/workflows/terraform.yml
name: Terraform CI

on:
  pull_request:
    paths: ['**/*.tf', '**/*.tfvars']

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.7.0

      - name: Terraform Format Check
        run: terraform fmt -check -recursive

      - name: Terraform Init
        run: terraform init -backend=false

      - name: Terraform Validate
        run: terraform validate

      - name: Terraform Plan
        run: terraform plan -no-color
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

## 3. Git Hooks

---

## Hooks Locais AUTOMAÇÃO

Scripts executados automaticamente em eventos Git.

```
# .git/hooks/pre-commit
#!/bin/bash

echo "Rodando lint antes do commit..."
npm run lint
if [ $? -ne 0 ]; then
    echo "❌ Lint falhou! Corrija antes de commitar."
    exit 1
fi

# .git/hooks/commit-msg
#!/bin/bash

MSG=$(cat "$1")
PATTERN="^(feat|fix|docs|style|refactor|test|chore)(\(.+\))?: .+"
if ! echo "$MSG" | grep -qE "$PATTERN"; then
    echo "❌ Mensagem fora do padrão Conventional Commits"
    exit 1
fi
```

## Husky + lint-staged FERRAMENTA

Gerencia hooks de forma compartilhável com o time.

```
# Instalar Husky
npm install -D husky lint-staged
npx husky init

# .husky/pre-commit
npx lint-staged

# package.json
{
  "lint-staged": {
    "*.js,ts": ["eslint --fix", "prettier --write"],
    "*.css,md": ["prettier --write"]
  }
}

# Agora todo commit roda lint automaticamente
# E é compartilhado via package.json!
```

 **Git & GitHub — Módulo 08**

# GitHub: Projects, Pages, Environments e Segurança

GitHub Projects, Pages, Environments, Branch Protection, CODEOWNERS, Dependabot. Recursos avançados para gestão de projetos e segurança.

## 1. Branch Protection Rules

### Branch Protection SEGURANÇA

Proteja branches importantes contra push direto e merges sem review.

```
# Settings → Branches → Add rule
# Branch name pattern: main

# Regras recomendadas:
✓ Require a pull request before merging
  ✓ Require approvals (mínimo 1)
  ✓ Dismiss stale pull request approvals
✓ Require status checks to pass
  ✓ Require branches to be up to date
  → Seleccione: CI Pipeline, Tests
✓ Require conversation resolution
✓ Require signed commits (opcional)
✓ Do not allow bypassing the above settings

# Resultado: ninguém faz push direto na main
# Todo código passa por PR + CI + Review
```

## CODEOWNERS

## GOVERNANÇA

Define responsáveis automáticos por review.

```
# .github/CODEOWNERS

# Donos globais (review em tudo)
* @tech-lead @senior-dev

# Por diretório
/src/auth/    @security-team
/infra/       @devops-team
/docs/        @tech-writer

# Por extensão
*.tf          @devops-team
*.yaml        @devops-team
*.test.ts     @qa-team

# Resultado: PRs que tocam esses arquivos
# automaticamente pedem review dos owners
```

**Environments** **DEPLOY**

Controle deploys com aprovações e secrets por ambiente.

```
# Settings → Environments

# Ambiente: production
# - Required reviewers: @tech-lead
# - Wait timer: 5 minutes
# - Deployment branches: main only
# - Secrets: AWS_ROLE_ARN, CF_DIST_ID

# Ambiente: staging
# - No protection rules
# - Deployment branches: develop, release/*
# - Secrets: AWS_ROLE_ARN_STAGING

# No workflow:
jobs:
  deploy:
    environment: production # ← ativa proteções
    runs-on: ubuntu-latest
```

## 2. GitHub Projects e Pages

---

## GitHub Projects GESTÃO

Kanban integrado ao repositório.

```
# Criar Project (V2):  
# Repo → Projects → New Project  
  
# Colunas sugeridas:  
# 📋 Backlog  
# 🚧 In Progress  
# 👁 In Review  
# ✅ Done  
  
# Automações:  
# - Issue criada → Backlog  
# - PR aberto → In Progress  
# - PR merged → Done  
  
# Campos customizados:  
# - Priority: P0, P1, P2  
# - Sprint: Sprint 1, Sprint 2  
# - Size: S, M, L, XL
```

## GitHub Pages DEPLOY

Hospede sites estáticos gratuitamente.

```
# Settings → Pages

# Source: GitHub Actions (recomendado)
# ou
# Source: Deploy from branch (gh-pages)

# Workflow para Pages:
# .github/workflows/pages.yml
name: Deploy to GitHub Pages
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      pages: write
      id-token: write
    environment:
      name: github-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/upload-pages-artifact@v3
        with:
          path: './dist'
      - uses: actions/deploy-pages@v4
```

## Segurança GITHUB

Recursos de segurança do repositório.

```
# Settings → Security

# Dependabot Alerts
# → Detecta vulnerabilidades em dependências
# → Cria PRs automáticos com fix

# Secret Scanning
# → Detecta tokens/keys commitados
# → Alerta e pode bloquear push

# Code Scanning (CodeQL)
# → Análise estática de segurança
# → Detecta SQL injection, XSS, etc.

# Security Policy
# → SECURITY.md na raiz
# → Como reportar vulnerabilidades

# Signed Commits (GPG)
git config --global commit.gpgsign true
git config --global user.signingkey KEY_ID
```

## Templates

### ORGANIZAÇÃO

Padronize Issues e PRs com templates.

```
# .github/ISSUE_TEMPLATE/bug_report.md
---
name: Bug Report
about: Reporte um bug
labels: bug
---
## Descrição
## Passos para Reproduzir
## Comportamento Esperado
## Screenshots
## Ambiente

# .github/PULL_REQUEST_TEMPLATE.md
## O que foi feito
## Tipo de mudança
- [ ] Bug fix
- [ ] Nova feature
- [ ] Breaking change
## Checklist
- [ ] Testes adicionados
- [ ] Documentação atualizada
- [ ] Lint passa sem erros
```